

AURALIS

Roadmap de desarrollo móvil con Flutter + Dart

Propósito: construir desde cero una aplicación móvil para gestionar y reproducir música almacenada localmente en el dispositivo, aprendiendo Flutter progresivamente y usando Codex como asistente de desarrollo.

Contexto: Auralis será inicialmente un proyecto independiente y local. Más adelante puede evolucionar hacia integración con servicios como Navidrome u otras fuentes de música.

Visión del proyecto

Auralis detectará la música disponible en el teléfono, leerá sus metadatos y la organizará en una interfaz moderna y responsive. Tendrá reproducción, biblioteca, favoritos, playlists, artistas, álbumes, búsqueda, historial y funciones avanzadas.

Stack inicial

Framework: Flutter

Lenguaje: Dart

Estado: Riverpod

Audio: just_audio + solución de background audio

Persistencia: SQLite/Drift en una etapa posterior

IDE: VS Code o Android Studio

Control de versiones: Git + GitHub

Primera plataforma objetivo: Android

Arquitectura propuesta

lib/

■■■ core/ – tema, navegación, constantes y utilidades

■■■ features/ – música, reproductor, playlists, favoritos, artistas, álbumes, búsqueda y configuración

■■■ data/ – modelos, repositorios y fuentes de datos

■■■ main.dart

La arquitectura se irá construyendo conforme aparezca la necesidad. No es necesario implementar todo desde el primer día.

Roadmap por sprints

SPRINT 0 — Preparación

- Instalar Flutter SDK, Dart, Android Studio/SDK y configurar VS Code o Android Studio.
- Crear el proyecto Flutter y entender pubspec.yaml, dependencias y estructura básica.
- Configurar Git/GitHub y realizar el primer commit.
- Aprender lo mínimo de Dart necesario para comenzar.

SPRINT 1 — Diseño de la aplicación

- Crear Home, Library, Playlists, Player y Settings.
- Aprender widgets, layouts, navegación, Material 3, temas, listas y responsive UI.
- Implementar dark mode y un mini player visual.
- La app todavía puede usar datos falsos.

SPRINT 2 — Detectar música del dispositivo

- Solicitar permisos necesarios.
- Detectar archivos de audio locales.
- Leer título, artista, álbum, género, duración, portada, ruta y formato.
- Crear el modelo Song y mostrar la biblioteca real.

SPRINT 3 — Reproductor básico

- Integrar el motor de audio.
- Implementar play, pause, next, previous, seek, progreso y duración.
- Conectar la UI con el estado del reproductor.
- Crear mini player y pantalla completa del reproductor.

SPRINT 4 — Reproducción avanzada

- Implementar cola de reproducción.
- Shuffle, repeat, repeat-one y navegación entre canciones.
- Reproducción en segundo plano.
- Controles desde notificación/pantalla de bloqueo cuando la plataforma y configuración lo permitan.

SPRINT 5 — Favoritos

- Marcar/desmarcar canciones como favoritas.
- Crear pantalla de favoritos.
- Persistir favoritos localmente.

SPRINT 6 — Playlists

- Crear, editar y eliminar playlists.
- Agregar/quitar canciones.
- Reordenar canciones.

- Mostrar portada, nombre y cantidad de canciones.

SPRINT 7 — Artistas y álbumes

- Organizar biblioteca por canciones, artistas, álbumes y géneros.
- Crear vistas de artista y álbum.
- Navegar desde artista → álbum → canciones.

SPRINT 8 — Búsqueda e historial

- Búsqueda global por canción, artista y álbum.
- Historial de reproducción.
- Recently Played, Recently Added y Most Played.

SPRINT 9 — Base de datos y persistencia

- Introducir SQLite/Drift.
- Persistir canciones, playlists, favoritos e historial.
- Separar modelos, repositorios y fuentes de datos.
- Optimizar la carga de bibliotecas grandes.

SPRINT 10 — Funciones avanzadas

- Sleep timer.
- Ecualizador, según soporte disponible.
- Letras, respetando APIs/licencias.
- Crossfade.
- Gapless playback.
- Estadísticas de escucha.
- Smart playlists.
- Sistema de rating.
- Detección de posibles duplicados.

SPRINT 11 — UX y optimización

- Loading, empty y error states.
- Cache de imágenes.
- Reducir reconstrucciones innecesarias.
- Revisar rendimiento, memoria y tiempo de inicio.
- Mejorar animaciones y transiciones sin sobrecargar la interfaz.

SPRINT 12 — Release

- Testing.
- Build de release.
- Pruebas en dispositivo físico.
- README con arquitectura, capturas y decisiones técnicas.

- Preparar APK/AAB y documentación del proyecto.

Niveles del proyecto

Nivel	Incluye
MVP	Detectar música, biblioteca, play/pause, siguiente/anterior y reproductor.
Aplicación completa	Favoritos, playlists, artistas, álbumes, búsqueda, historial, cola, shuffle, repeat, background playback y persistencia.
Avanzado	Ecualizador, letras, estadísticas, smart playlists, sleep timer, crossfade, gapless, detección de duplicados, testing.

Orden recomendado para aprender Flutter

Dart: variables, tipos, funciones, clases, constructores, null safety, colecciones, Future, async/await y Streams.

Flutter: widgets, StatelessWidget, StatefulWidget, BuildContext, layouts, Row, Column, Stack, ListView y navegación.

Arquitectura: estado, Riverpod, modelos, servicios, repositorios y separación de responsabilidades.

Proyecto: aplicar inmediatamente cada concepto en Auralis en lugar de intentar estudiar Flutter completo antes de programar.

Cómo trabajar con Codex

- Mantener este documento como referencia general del proyecto.
- Trabajar un sprint a la vez y evitar implementar funciones de sprints posteriores sin necesidad.
- Pedir a Codex que explique los cambios importantes y la razón de cada decisión técnica.
- Revisar el código generado y tratar de entenderlo antes de continuar.
- Hacer commits pequeños y descriptivos.
- Cuando aparezca un error, primero leer el mensaje, identificar el archivo afectado y luego pedir ayuda con el contexto completo.
- No convertir Codex en una caja negra: el objetivo es aprender Flutter mientras se construye Auralis.

Entorno de desarrollo y pruebas

Flutter no usa Expo Go. Expo Go pertenece al ecosistema de React Native/Expo. En Flutter normalmente ejecutas la aplicación directamente con Flutter sobre un dispositivo físico o un emulador.

Android Studio: además de ser un IDE opcional para programar, instala y administra herramientas importantes del ecosistema Android, como el Android SDK y el Android Emulator.

Emulador: sí, sirve para probar la app sin tener necesariamente un teléfono Android conectado. Puedes crear un dispositivo virtual y ejecutar Auralis allí.

Teléfono físico: también puedes conectar un Android mediante USB, habilitar las opciones de desarrollador y depuración USB y ejecutar la aplicación directamente en el teléfono. Para una app que trabaja con archivos de música, las pruebas en dispositivo físico serán especialmente importantes.

Flujo típico:

Código → Flutter → dispositivo físico o emulador → pruebas → correcciones → commit

Android será la plataforma inicial recomendada. Una adaptación posterior a iOS puede estudiarse cuando el reproductor y la biblioteca local estén estables.

Evolución futura

Una vez terminada la versión local, Auralis puede convertirse en una segunda etapa de aprendizaje e integrarse con una fuente como Navidrome, siempre revisando la API y autenticación que corresponda. Esto permitiría explorar música local + servidor, sincronización, cuentas y otras capacidades sin mezclar esas complejidades en el MVP.

Objetivo final: una aplicación móvil de música funcional, bien estructurada y suficientemente completa para demostrar conocimientos reales de Flutter, Dart, arquitectura, estado, persistencia, audio y desarrollo Android.